

UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Course Name: Operating System Lab

Course Code: CSE 402

LAB REPORT

Lab Report Number : 05

Lab Report Name : SJF (Shortest Job First) Preemptive Scheduling Algorithm

Date of Submission : 23.08.2026

Submitted By:

Name : Ejaj Ahmmod Lemon

ID : 23101160

Section: C2

Submitted To:

Atia Rahman Orthi

Lecturer

University of Asia Pacific

1. Problem Statement:

An application is to be developed to implement the **Shortest Job First–Preemptive (SJF-P)** CPU Scheduling Algorithm. In this scheduling technique, the process with the **shortest remaining burst time** among all processes that have already arrived and are ready for execution is selected for execution.

Since the algorithm is **preemptive**, the currently executing process may be interrupted whenever a new process arrives with a shorter remaining burst time than the currently running process. The preempted process is returned to the ready queue and resumes execution later when it again has the shortest remaining burst time.

The processes may have different **Arrival Times (AT)** and **Burst Times (BT)**. A **Time Quantum (TQ) of 2** is considered for the given test case. A process can be scheduled only after it has arrived in the ready queue. The application must continuously compare the remaining burst times of all available processes and determine the appropriate execution order.

For the given test case, **five processes (P1–P5)** are considered with a **Time Quantum (TQ) = 2** and the following Arrival Times and Burst Times:

PID	Arrival Time (AT)	Burst Time (BT)
P1	4	2
P2	2	2
P3	1	3
P4	0	6
P5	3	1

The application must calculate the required performance measures, including **Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT)** for each process. It should also determine the **Average Turnaround Time** and **Average Waiting Time**, and display the **Gantt Chart** showing the execution sequence of the processes.

Given Time Quantum: TQ = 2

2. Objectives:

The main objectives of implementing the **Shortest Job First–Preemptive (SJF-P)** CPU Scheduling Algorithm are:

1. **To understand the operational principles** of the Shortest Job First–Preemptive CPU scheduling algorithm.
2. **To implement the preemptive SJF scheduling algorithm** by selecting the process with the shortest remaining burst time among the available processes.
3. **To handle processes with different Arrival Times (AT)** and Burst Times (BT) correctly.
4. **To implement process preemption** when a newly arrived process has a shorter remaining burst time than the currently executing process.
5. **To calculate Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT)** for each process.
6. **To calculate and analyze the Average Turnaround Time and Average Waiting Time** to evaluate scheduling performance.
7. **To construct and display the Gantt Chart** showing the execution sequence and preemption of processes.
8. **To analyze the effect of the Time Quantum (TQ = 2)** used in the given test case and observe its impact on process execution.
9. **To understand the advantages and limitations** of SJF Preemptive scheduling, particularly its ability to reduce average waiting time while potentially causing starvation of longer processes.

3. Source Code Screenshot:

```python
TQ = 2
processes = [
    {"pid": "P1", "at": 4, "bt": 2},
    {"pid": "P2", "at": 2, "bt": 2},
    {"pid": "P3", "at": 1, "bt": 3},
    {"pid": "P4", "at": 0, "bt": 6},
    {"pid": "P5", "at": 3, "bt": 1}
]

for p in processes:
    p["rem"] = p["bt"]
    p["ct"] = 0
    p["tat"] = 0
    p["wt"] = 0
time = 0
completed = 0
n = len(processes)
gantt = []

while completed < n:
    available = [
        p for p in processes
        if p["at"] <= time and p["rem"] > 0
    ]
    if not available:
        time += 1
        continue
    current = min(
        available,
        key=lambda p: (p["rem"], p["at"], p["pid"])
    )
    run_time = min(TQ, current["rem"])
    start = time
    time += run_time
    current["rem"] -= run_time
    gantt.append((start, time, current["pid"]))

    if current["rem"] == 0:
        current["ct"] = time
        current["tat"] = current["ct"] - current["at"]
        current["wt"] = current["tat"] - current["bt"]
        completed += 1

print("\nSJF Preemptive")
print("Time Quantum =", TQ)
print("\nPID\tAT\tBT\tCT\tTAT\tWT")
print("-" * 50)
total_tat = 0
total_wt = 0

for p in sorted(processes, key=lambda x: x["pid"]):
    print(
        f'{p["pid"]}\t'
        f'{p["at"]}\t'
        f'{p["bt"]}\t'
        f'{p["ct"]}\t'
        f'{p["tat"]}\t'
        f'{p["wt"]}'
    )
    total_tat += p["tat"]
    total_wt += p["wt"]
print("-" * 50)
print(f"Average TAT = {total_tat / n:.2f}")
print(f"Average WT  = {total_wt / n:.2f}")
print("\nGantt Chart:")
for start, end, pid in gantt:
    print(f"{start} -> {end}: {pid}")
```

Figure 1: Source Code of the SJF Preemptive Algorithm

4. Output Screenshot:

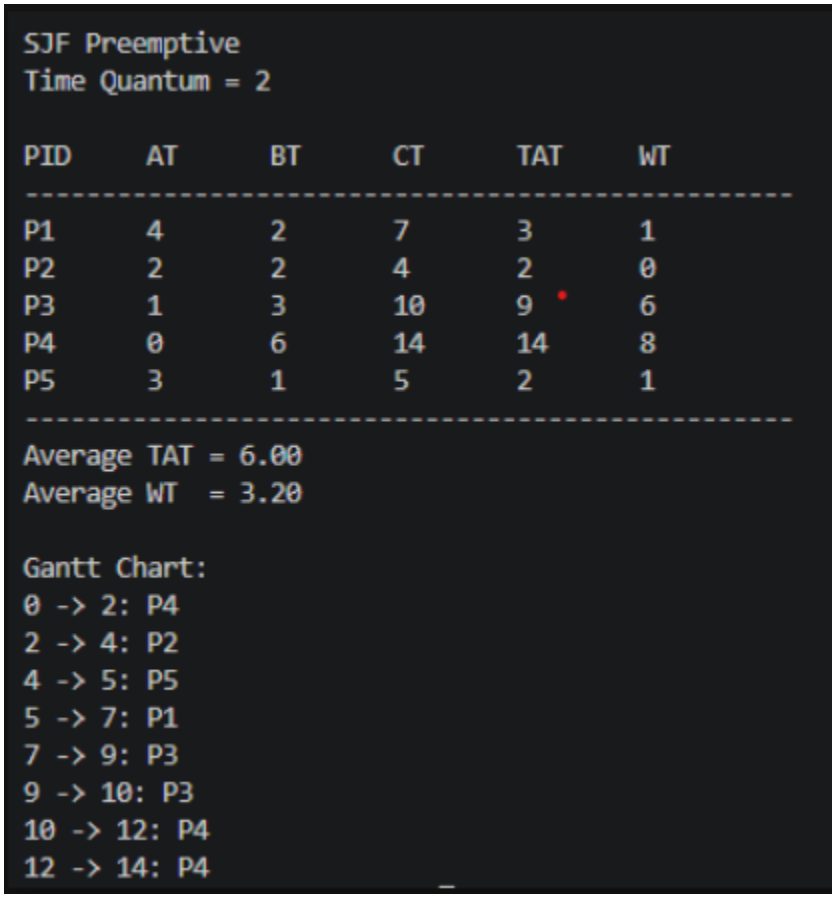


Figure 2: Execution Output of the **SJF Preemptive** Algorithm

The program successfully executes the SJF Preemptive CPU Scheduling Algorithm for the given processes with Time Quantum (TQ) = 2. It displays the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time. The Gantt Chart shows the execution sequence of the processes according to their shortest remaining burst time.

5. Discussion:

The **Shortest Job First–Preemptive (SJF-P)** scheduling algorithm was successfully implemented and tested with the given processes. The algorithm selects the process with the **shortest remaining burst time**, allowing a newly arrived shorter process to preempt the currently running process. The results show that preemption helps reduce the waiting time of shorter processes and improves CPU scheduling efficiency. The **Completion Time, Turnaround Time, and Waiting Time** were calculated for each process, and the Gantt Chart clearly represents the execution sequence. However, SJF-P may cause **starvation for processes with longer burst times** if shorter processes continue to arrive. Overall, the algorithm is effective for workloads where minimizing average waiting and turnaround time is important.

6. Conclusion:

The Shortest Job First–Preemptive (SJF-P) CPU scheduling algorithm was successfully implemented and analyzed. The algorithm dynamically selects the process with the shortest remaining burst time and can preempt the currently running process when a shorter process arrives. The required scheduling parameters, including Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT), were calculated successfully. The experiment demonstrates that SJF-P can provide efficient CPU utilization and generally achieve a lower average waiting time by giving priority to shorter processes. However, frequent preemption can increase context-switching overhead and may cause starvation of longer processes.

In general, SJF Preemptive is better when the main goal is to reduce waiting time and improve responsiveness, because a newly arrived shorter process can immediately get CPU time.

Feature	SJF Preemptive	SJF Non-Preemptive
Preemption	Yes	No
Process selection	Shortest remaining time	Shortest burst time
Waiting time	Usually lower	Usually higher
Response to new short process	Faster	Must wait for current process
Context switching	More	Less
Implementation	More complex	Simpler
Starvation	Possible	Possible
Suitable for	Interactive/dynamic systems	Batch systems

Conclusion of comparison:
SJF Preemptive (SRTF) is generally more efficient than SJF Non-Preemptive in terms of average waiting time and response time, especially when processes arrive at different times. However, SJF Non-Preemptive is simpler and has less context-switching overhead. Therefore, SJF Preemptive is preferable when performance and responsiveness are more important, while SJF Non-Preemptive is preferable when simplicity and lower overhead are the priorities.